EVIE Demand Dashboard Data Creation Guide

1 Purpose

This document explains how to create the input data files used by the EVIE demand dashboard. The dashboard itself reads four CSV files:

- `clean_trips_with_invoice_plan_and_segment.csv`
- `user_month_regular.csv`
- `user_month_student.csv`
- `user_month_access.csv`

The first file is trip-level. The three user-month files define the demand markets and observed choices for each user segment. These files are produced by two scripts in `Dashboard/DataPrep/`:

1. `CleanTrips.py`
2. `CreateUserMonthData.py`

Run the scripts in that order. The scripts are designed to be readable and to print diagnostics at each step. Those printed diagnostics should be reviewed whenever new raw data are processed.

2 Folder Setup

Place the raw company exports in `Dashboard/DataPrep/`. By default, `CleanTrips.py` expects:

- `Trips_2026-06-08.csv`
- `Invoices_2026-06-08.csv`

If new raw files have different names, edit the settings at the top of `CleanTrips.py`:

```
TRIPS_CSV_NAME = "Trips_YYYY-MM-DD.csv"
INVOICES_CSV_NAME = "Invoices_YYYY-MM-DD.csv"
```

The scripts resolve paths relative to their own folder. This means they can be run from an IDE, from a terminal, or by double-checking the working directory less carefully than a normal relative-path script.

3 Script 1: CleanTrips.py

3.1 Purpose

`CleanTrips.py` creates the trip-level dashboard input file. It cleans the raw trips export, attaches invoice plan information, assigns each user to a segment, and attaches posted pricing parameters at the time of each trip.

3.2 Required Raw Trip Columns

The trip export must include:

Column	Use
`TripID`	Unique trip identifier used to join invoices.
`UserID`	User identifier.
`StartDateUTC`	Trip start time.
`TripDistance_mi`	Trip distance in miles.
`TripDuration`	Trip duration in minutes.
`ServiceType`	Used to keep free-floating trips.
`IsCancel`	Used to drop canceled trips.

3.3 Required Raw Invoice Columns

The invoice export must include:

Column	Use
`TripID`	Trip identifier used to join trips.
`MobilityPlan`	Raw plan name billed on the invoice.

3.4 Trip Filters

The script applies these filters:

1. require nonmissing `TripID`, `UserID`, and valid trip start time;
2. keep only trips with `ServiceType == FREE_FLOATING`;
3. drop canceled trips;
4. require trip duration of at least `MIN_DURATION_MINUTES`;
5. require trip distance of at least `MIN_DISTANCE_MILES`;
6. drop duplicate `TripID` rows in the trip file, keeping the earliest trip-start row.

The current minimum duration is 3 minutes and the current minimum distance is 0.5 miles. These settings are at the top of `CleanTrips.py`.

3.5 Invoice Plan Cleaning

Raw invoice plan names are normalized into:

- `Free Plan`
- `Everyday`
- `Access`
- `Student`

The script treats Adventure, error, missing, and unknown invoice plans as unusable for this demand model. It also removes "Plus" labels before classification. For example, plan names containing "PLUS", "plus", "+PLUS", or "+plus" are stripped back to their base plan name before normalization. This means Plus variants are not modeled as separate products.

After normalization, a trip is kept only if its `TripID` has exactly one usable normalized plan. Trips with no usable plan or conflicting usable plans are dropped.

3.6 User Segment Assignment

Segments are assigned from all usable invoice plans observed for each user:

- `Regular`: user only used `Free Plan` and/or `Everyday`.
- `Access`: user only used `Free Plan` and/or `Access`, and used `Access` at least once.
- `Student`: user only used `Free Plan` and/or `Student`, and used `Student` at least once.
- `Conflicting`: all other observed plan combinations.

The trip-level file keeps the `Conflicting` label for diagnostics, but the user-month panel script drops conflicting users before model input files are created.

3.7 Trip-Level Posted Pricing

For every clean trip, the script attaches two JSON strings:

- `PaidPlanPricingAtTrip`
- `FreePlanPricingAtTrip`

The paid plan is segment-specific:

- Regular: Everyday
- Access: Access
- Student: Student

Each pricing JSON includes:

- pricing regime label;
- plan name;
- per-minute rate;
- per-hour rate;
- per-day rate;
- included miles;
- extra-mile fee;
- trip start fee;

- monthly fee;
- monthly credit.

Trips before the first known posted pricing regime receive `UNKNOWN_POSTED_PRICE`. Conflicting users receive `CONFLICTING_USER_SEGMENT` for paid-plan pricing.

3.8 CleanTrips.py Output

The main output is:

```
Dashboard/DataPrep/01_clean_trips_invoice/
    clean_trips_with_invoice_plan_and_segment.csv
```

If `WRITE_DASHBOARD_COPY=True`, the same file is also written to:

```
Dashboard/clean_trips_with_invoice_plan_and_segment.csv
```

The output columns are:

Column	Meaning
`TripID`	Trip identifier.
`UserID`	User identifier.
`UserSegment`	Regular, Access, Student, or Conflicting.
`TripStartDT`	Parsed trip start timestamp.
`YearMonth`	Trip month.
`MonthOfYear`	Calendar month number.
`TripDurationMinutes`	Clean trip duration.
`TripDistanceMiles`	Clean trip distance.
`PlanUsedForTrip`	Normalized invoice plan used for that trip.
`PaidPlanPricingAtTrip`	Segment paid-plan pricing JSON.
`FreePlanPricingAtTrip`	Free Plan pricing JSON.

4 Script 2: CreateUserMonthData.py

4.1 Purpose

`CreateUserMonthData.py` converts the clean trip file into monthly user panels. These panels define the markets, market sizes, outside option, observed trip counts, observed plan choices, and monthly pricing parameters used by the demand model.

4.2 Input

The script reads:

```
Dashboard/DataPrep/01_clean_trips_invoice/
    clean_trips_with_invoice_plan_and_segment.csv
```

Run `CleanTrips.py` before running this script.

4.3 Dropping Conflicting Users

Users classified as `Conflicting` are dropped. This keeps the final segment panels mutually exclusive and avoids users whose observed plan history does not map cleanly to one paid-plan segment.

4.4 Output Month Range

The current settings start output panels in 2023-04:

```
FIRST_OUTPUT_MONTH = "2023-04"
```

If `LAST_OUTPUT_MONTH=None`, the last output month is the last month present in the clean trip file.

4.5 Full User-Month Grid

The script creates a row for every non-conflicting user and every output month. This is important because users with zero trips still contribute to market size and the outside option when they are in market.

4.6 Market Membership

`InMarket` indicates whether a user is included in the market for a month. The current rule is:

$$InMarket_{it} = 1 \quad if user i used EVIE in month t or in the prior K months. \tag{1}$$

The current setting is:

```
K_MONTHS_IN_MARKET = 12
INCLUDE_CURRENT_MONTH_USE = True
```

Thus a user is in market if they used EVIE in the current month or in the previous 12 months. This rule defines market size and therefore defines the outside option.

4.7 Monthly Usage Aggregation

For each user-month, the script computes:

- number of trips;
- total monthly trip distance;
- total monthly trip duration;
- monthly plan used.

For positive-use months, `PlanUsedThisMonth` is the most-used trip plan in that month. If there is a tie, the segment paid plan is preferred over Free Plan. For zero-trip in-market months, `PlanUsedThisMonth=Free Plan`. For zero-trip not-in-market months, `PlanUsedThisMonth=Not In Market`.

4.8 Monthly Posted Pricing

Monthly pricing is attached for each segment-month. If a month contains a price change during the month, the script assigns the pricing regime active for the largest number of days in that month. The output includes both the segment paid plan pricing JSON and the Free Plan pricing JSON.

4.9 CreateUserMonthData.py Outputs

The main outputs are written to:

`Dashboard/DataPrep/02_user_month_panels/`

The files are:

- `user_month_all_segments.csv`
- `user_month_regular.csv`
- `user_month_access.csv`
- `user_month_student.csv`

If `WRITE_DASHBOARD_COPIES=True`, the segment-specific files are also written to the main `Dashboard/` folder, where the dashboard reads them.

The output columns are:

Column	Meaning
`UserID`	User identifier.
`UserSegment`	Regular, Access, or Student.
`YearMonth`	User-month.
`MonthOfYear`	Calendar month number.
`InMarket`	1 if user is included in the market that month.
`TripCount`	Number of EVIE trips in the month.
`TotalDistanceMiles`	Total monthly trip distance.
`TotalDurationMinutes`	Total monthly trip duration.
`PlanUsedThisMonth`	Monthly plan choice used for product definition.
`PricingRegime_Month`	Posted-pricing regime assigned to the month.
`PaidPlanName`	Segment paid-plan name.
`PaidPlanPricingAtMonth`	Paid-plan monthly pricing JSON.
`FreePlanPricingAtMonth`	Free Plan monthly pricing JSON.

5 Adding A Future Price Change

Pricing rules appear in both scripts because one script assigns trip-level prices and the other assigns month-level prices. When EVIE changes prices:

1. Update `CleanTrips.py`.
2. Update `CreateUserMonthData.py`.
3. In each script, add a new `EVIE_RULES_START_*` timestamp.
4. Add a new entry to `EVIE_PRICING_RULES`.
5. Include all four plans: Everyday, Access, Student, and Free Plan.
6. Include all tariff keys: `per_minute`, `per_hour`, `per_day`, `included_miles`, `extra_mile`, `trip_start_fee`, `monthly_fee`, and `monthly_credit`.

7. Add the new regime check at the top of `pricing_regime_at_trip()` in `CleanTrips.py`.

8. Add the new regime check at the top of `pricing_regime_at_day()` in `CreateUserMonthData.py`.

The newest pricing regime must be checked first because the functions use "if date is after start date" logic.

6 Recommended Refresh Procedure

For a new raw-data delivery:

1. Copy the new raw trip and invoice exports into `Dashboard/DataPrep/`.
2. Update `TRIPS_CSV_NAME` and `INVOICES_CSV_NAME` in `CleanTrips.py`.
3. If EVIE prices changed, update pricing rules in both scripts.
4. Run `CleanTrips.py`.
5. Review printed diagnostics: dropped trips, missing invoice plans, segment counts, and pricing special values.
6. Run `CreateUserMonthData.py`.
7. Review printed diagnostics: conflicting users, market membership, plan counts, pricing regimes, and output row counts.
8. Confirm the four dashboard input files exist in the main `Dashboard/` folder.
9. Open the dashboard and use the Model Validation tab to verify the refreshed data before using what-if or optimization tools.

7 Quality Checks

Before using refreshed data, check the following:

- The latest month is complete or is excluded in dashboard estimation.
- The number of dropped trips is reasonable relative to past refreshes.
- Normalized plan counts are plausible.
- Conflicting users are a small and explainable group.
- Pricing regimes by month match EVIE's known price-change dates.
- Segment-specific user-month files have nonzero rows.
- The dashboard loads without missing-file errors.